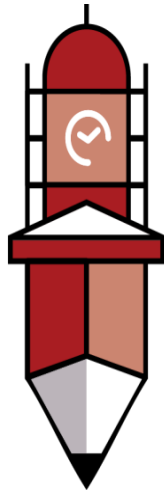


LAB MANUAL



The University of Faisalabad

SUBMITTED TO:	Mr. Arham
SUBMITTED BY:	Rihab Ahmed
REGISTRATION NO:	2023-BS-AI-177
SUBJECT:	Data Mining
COURSE CODE:	DS-303
CLASS:	BS-AI
SEMESTER:	6th (A)
DEPARTMENT:	CS (AI)

Department of Computer Science

The University of Faisalabad

Table of Contents

1. LAB 1: Introduction to Data Mining.....	1
2. LAB 2: CRISP-DM Process.....	9
3. LAB 3: Handling Missing Values, Duplicates & Noise.....	23
4. LAB 4: Normalization, Standardization & Discretization.....	31
5. LAB 5: Market Basket Analysis.....	42
6. LAB 6: Apriori Algorithm.....	59
7. LAB 7: FP-Tree Construction & FP-Growth.....	72
8. LAB 8: Python Syntax & Basics.....	79
9. LAB 9: Decision Tree Classifier.....	89
10. LAB 10: Naïve Bayes & KNN.....	97
11. LAB 11: Support Vector Machine (SVM).....	102
12. LAB 12: K-Means & Hierarchical Clustering.....	107
13. LAB 13: Outlier Detection.....	113

14. LAB 14: Data Scraping & Sentiment Analysis.....	120
15. LAB 15: Performance Comparison of Models.....	128

LAB 1: Introduction to Data Mining

Introduction

Modern retail businesses generate large amounts of sales data every day through customer purchases and inventory transactions. While this data contains valuable information, it has little practical use unless it is analyzed effectively.

Data mining helps organizations convert raw data into meaningful insights. By examining sales records, businesses can measure performance, identify trends, and support decision-making. In this lab, students will use retail transaction data to calculate important Key Performance Indicators (KPIs) such as Total Revenue, Average Order Value, Revenue by Category, and Top-Selling Categories.

The analysis will be performed using the KNIME Analytics Platform, a visual workflow-based tool that enables users to process and analyze data without extensive programming.

Problem Statement

A retail company has collected thousands of transaction records but lacks an efficient way to evaluate business performance. Management needs a structured method to analyze sales data and generate meaningful business metrics.

The goal is to build a KNIME workflow that processes transaction data and produces key insights, including total sales revenue, average transaction value, category performance, and product demand.

Dataset Description

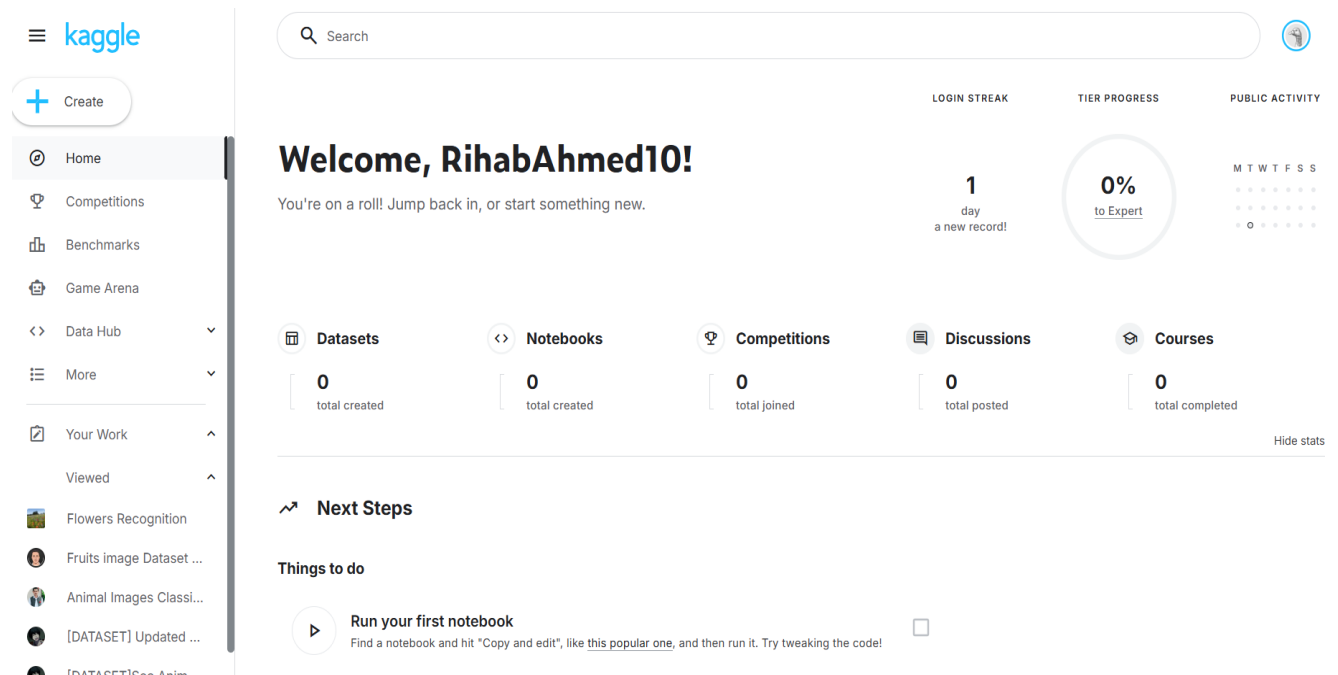
This lab uses a Retail Sales Dataset stored in CSV format. Each row represents a product purchased within a customer transaction.

Important Attributes

- **Transaction ID** – Unique identifier for each customer transaction.
- **Quantity** – Number of units purchased.

- **Product Category** – Category to which the product belongs (e.g., Electronics, Clothing, Beauty).
- **Price per Unit** – Cost of one product unit.

Although the dataset includes a **Total Amount** field, students will create a new **Revenue** column by multiplying **Quantity** × **Price per Unit** to practice feature engineering techniques.



Tool Used

KNIME Analytics Platform

KNIME is an open-source data analytics platform that uses a visual workflow approach. Instead of writing code, users connect nodes to perform tasks such as importing data, transforming values, calculating metrics, and generating reports.

Advantages of KNIME

- Easy-to-understand visual workflows.
- No programming syntax required.
- Flexible node configuration for data processing and analysis.
- Reproducible and transparent analytical processes.

Practical Objectives

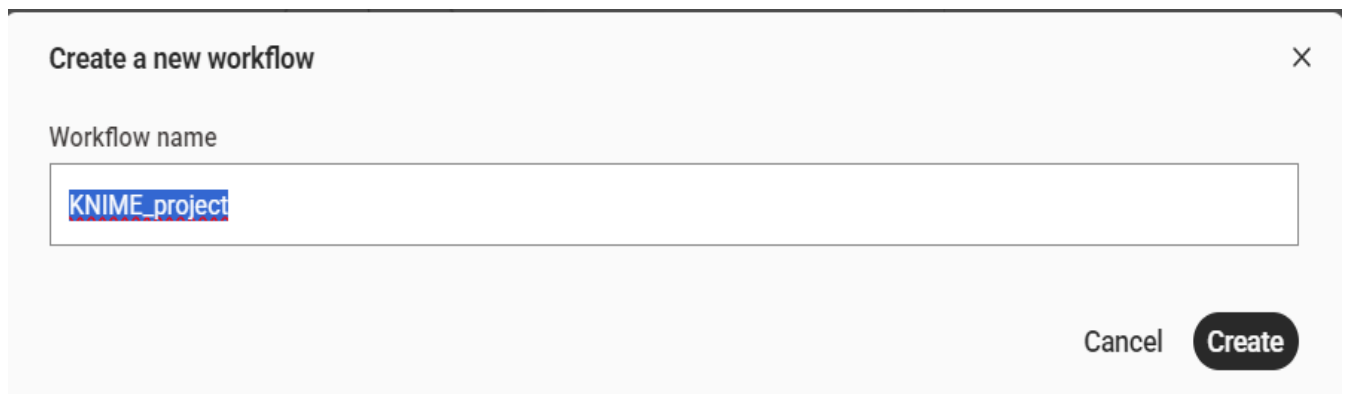
After completing this lab, students will be able to:

1. Import CSV data into KNIME.
2. Verify and correct data types.
3. Generate descriptive statistics.
4. Create a Revenue column using mathematical operations.
5. Calculate business KPIs using aggregation and sorting techniques.

Procedure

Step 1: Create a New Workflow

1. Open KNIME Analytics Platform.
2. Select **File** → **New** → **New KNIME Workflow**.
3. Name the workflow **Retail Sales Analysis**.
4. Click **Finish** to open a blank workspace.



Step 2: Import the Dataset

1. Drag a **CSV Reader** node onto the workflow.
2. Open its configuration window.

3. Browse and select the Retail Sales CSV file.
4. Apply the settings and execute the node.
5. View the imported data to confirm successful loading.



Step 3: Verify Data Types

Check that:

- Quantity → Integer
- Price per Unit → Number/Double
- Total Amount → Number/Double
- Date → Date format
- Product Category → String

If necessary, use:

- **String to Number** node for numeric conversion.
- **String to Date&Time** node for date conversion.

Step 4: Generate Summary Statistics

1. Add a **Statistics** node.
2. Connect it to the dataset.
3. Execute the node.
4. Review measures such as:

- Mean
- Minimum
- Maximum
- Standard Deviation

This helps understand the dataset before further analysis.

Statistics

Open in new window

No data | Columns: 14

Name	Type	# Missing val...	# Unique valu...	Minimum	Maximum	25% Quantile	50% Quantile ...	75% Quantile
------	------	------------------	------------------	---------	---------	--------------	------------------	--------------

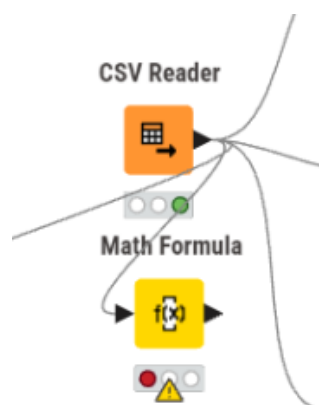
Step 5: Create the Revenue Column

1. Add a **Math Formula** node.
2. Connect it to the dataset.
3. Enter the formula:

Quantity $\times$ Price per Unit

4. Create a new column named **Calculated Revenue**.
5. Execute the node.

The new column will store the revenue generated by each transaction.

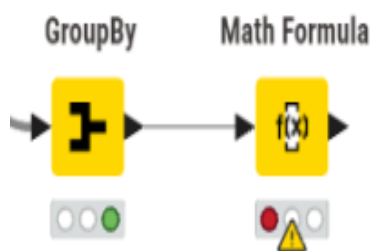


Step 6: KPI Calculations

KPI 1: Total Revenue

1. Connect a **GroupBy** node to the Math Formula node.
2. Leave the grouping section empty.
3. Apply **Sum** on the Calculated Revenue column.
4. Execute the node.

Result: Total revenue generated from all transactions.



KPI 2: Average Transaction Value

1. Add another **GroupBy** node.
2. Leave grouping empty.
3. Apply **Mean** on Calculated Revenue.
4. Execute.

Result: Average revenue per transaction.

KPI 3: Revenue by Category

1. Add a **GroupBy** node.
2. Group by **Product Category**.
3. Apply **Sum** on Calculated Revenue.
4. Connect a **Sorter** node.

5. Sort revenue in descending order.

Result: Categories ranked by revenue generated.

KPI 4: Top Categories by Sales Volume

1. Add another **GroupBy** node.
2. Group by **Product Category**.
3. Apply **Sum** on the Quantity column.
4. Connect a **Sorter** node.
5. Sort in descending order.

Result: Categories ranked by total units sold.

Summary

KPI	Purpose	KNIME Nodes
Total Revenue	Total sales generated	Math Formula → GroupBy (Sum)
Average Transaction Value	Average revenue per transaction	Math Formula → GroupBy (Mean)
Revenue by Category	Revenue generated by each category	Math Formula → GroupBy → Sorter
Top Categories by Volume	Categories with highest quantity sold	Math Formula → GroupBy → Sorter

Conclusion

This lab demonstrates how KNIME can be used to transform raw retail transaction data into meaningful business insights. By creating workflows and calculating KPIs, students gain practical experience in data preparation, feature engineering, aggregation, and business analytics.

LAB 2: CRISP-DM Process

Introduction

Customer retention is a major factor in the success of modern businesses. Acquiring new customers often costs more than keeping existing ones, making customer churn a significant challenge. Churn occurs when customers stop using a company's products or services.

Data mining techniques can help organizations identify customers who are likely to leave by analyzing historical customer information and behavior patterns. These insights allow businesses to take preventive actions and improve customer retention.

In this lab, students will apply the **CRISP-DM (Cross-Industry Standard Process for Data Mining)** methodology to build and evaluate a customer churn prediction model using Orange Data Mining.

Problem Statement

A company is experiencing customer loss but has no reliable method for identifying customers who may leave in the future. As a result, retention efforts are often delayed and ineffective.

The objective is to develop a predictive model that can classify customers as likely to churn or remain active. This will enable the business to implement targeted retention strategies and reduce revenue loss.

Dataset Description

This lab uses the **Customer Churn Dataset (Churn_Modelling.csv)**, which contains customer demographic, financial, and account activity information.

Key Attributes

Identification Fields

- RowNumber
- CustomerId
- Surname

These fields are identifiers and are not useful for prediction.

Demographic Information

- Geography
- Gender
- Age

Financial Information

- CreditScore
- Balance
- EstimatedSalary

Customer Activity Information

- Tenure
- NumOfProducts

- HasCrCard
- IsActiveMember

Target Variable

- **Exited**
 - 1 = Customer Left
 - 0 = Customer Retained

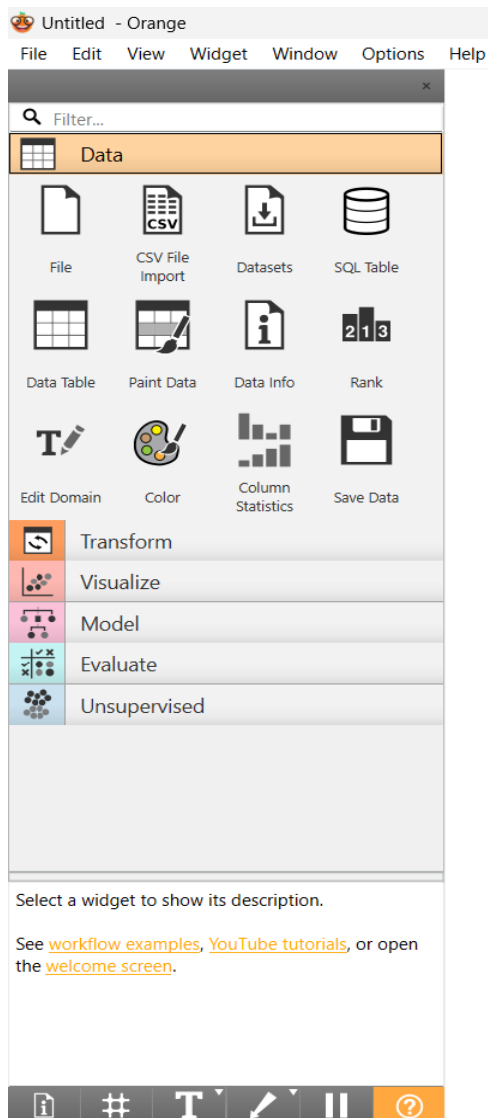
Tool Used

Orange Data Mining

Orange is an open-source visual data mining platform that allows users to build machine learning workflows through drag-and-drop widgets. It simplifies data analysis and model development without requiring extensive programming knowledge.

Benefits

- Easy-to-use graphical interface
- Rapid machine learning workflow creation
- Interactive visualizations
- Supports the complete CRISP-DM process



Learning Objectives

After completing this lab, students will be able to:

1. Apply the CRISP-DM framework to a real-world business problem.
2. Import and explore customer data.
3. Prepare data for machine learning.
4. Build a customer churn prediction model.
5. Evaluate model performance using business and machine learning metrics.

Procedure

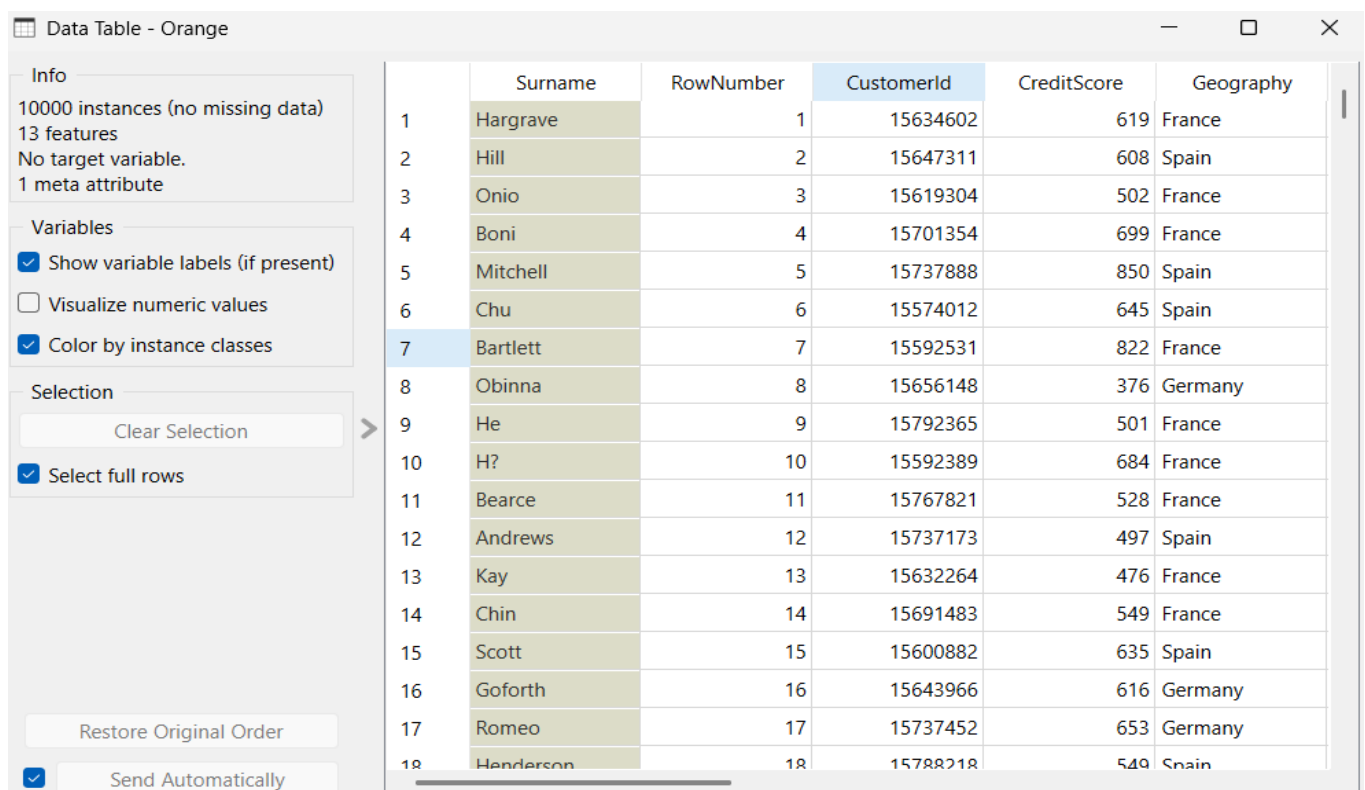
Step 1: Load the Dataset

Purpose

Import the customer churn data into Orange.

Steps

1. Open Orange Data Mining.
2. Drag a **File** widget onto the canvas.
3. Select **Churn_Modelling.csv**.
4. Add a **Data Table** widget.
5. Connect File → Data Table.
6. Verify that the dataset loads correctly.



The screenshot shows the 'Data Table - Orange' window. On the left, there is a sidebar with settings: 'Info' (10000 instances, 13 features, no target variable, 1 meta attribute), 'Variables' (Show variable labels, Visualize numeric values, Color by instance classes), and 'Selection' (Clear Selection, Select full rows). At the bottom, there are buttons for 'Restore Original Order' and 'Send Automatically' (checked). The main area displays a table with 18 rows and 6 columns: RowNumber, Surname, CustomerId, CreditScore, and Geography. The 'CustomerId' column is highlighted in blue.

	Surname	RowNumber	CustomerId	CreditScore	Geography
1	Hargrave	1	15634602	619	France
2	Hill	2	15647311	608	Spain
3	Onio	3	15619304	502	France
4	Boni	4	15701354	699	France
5	Mitchell	5	15737888	850	Spain
6	Chu	6	15574012	645	Spain
7	Bartlett	7	15592531	822	France
8	Obinna	8	15656148	376	Germany
9	He	9	15792365	501	France
10	H?	10	15592389	684	France
11	Bearce	11	15767821	528	France
12	Andrews	12	15737173	497	Spain
13	Kay	13	15632264	476	France
14	Chin	14	15691483	549	France
15	Scott	15	15600882	635	Spain
16	Goforth	16	15643966	616	Germany
17	Romeo	17	15737452	653	Germany
18	Henderson	18	15788218	540	Spain

Step 2: Explore the Data

Purpose

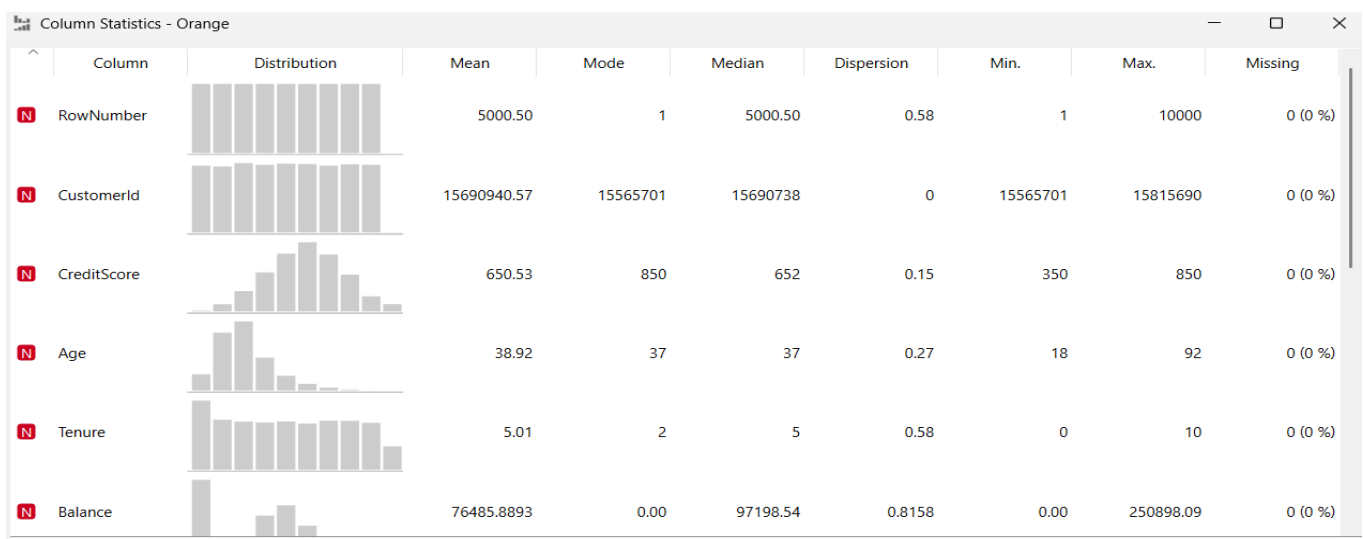
Understand data distributions and identify important characteristics.

Steps

1. Add a **Feature Statistics** widget.
2. Connect File → Feature Statistics.
3. Open the widget.
4. Examine distributions of:
 - Age
 - CreditScore
 - Balance
 - Exited

Outcome

Gain insight into customer characteristics and churn distribution.



Step 3: Select Features and Define Target

Purpose

Prepare the dataset for machine learning.

Steps

1. Add a **Select Columns** widget.
2. Connect File → Select Columns.
3. Set:
 - **Target:** Exited
 - **Ignored:** RowNumber, CustomerId, Surname
 - **Features:** All remaining variables
4. Apply changes.
5. Connect a Data Table to verify the configuration.

Outcome

The dataset is cleaned and ready for model training.

	Exited	CreditScore	Geography	Gender	Age
1	1	619	France	Female	42
2	0	608	Spain	Female	41
3	1	502	France	Female	42
4	0	699	France	Female	39
5	0	850	Spain	Female	43
6	1	645	Spain	Male	44
7	0	822	France	Male	50
8	1	376	Germany	Female	29
9	0	501	France	Male	44
10	0	684	France	Male	27
11	0	528	France	Male	31
12	0	497	Spain	Male	24
13	0	476	France	Female	34
14	0	549	France	Female	25
15	0	635	Spain	Female	35
16	0	616	Germany	Male	45
17	1	653	Germany	Male	58
18	0	540	Spain	Female	24

Step 4: Create the Prediction Model

Purpose

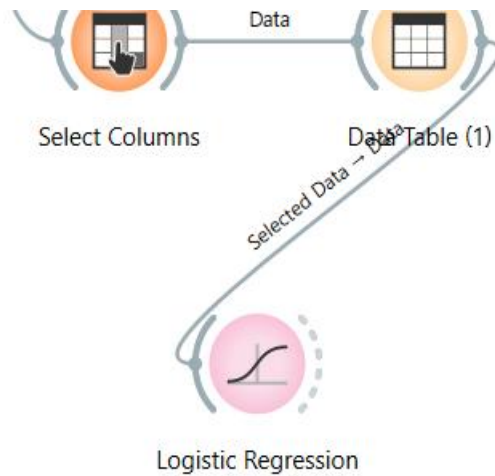
Initialize a classification algorithm.

Steps

1. Add a **Logistic Regression** widget.
2. Open its settings.
3. Keep the default parameters.
4. Close the configuration window.

Outcome

A machine learning model is ready for evaluation.



Step 5: Evaluate the Model

Purpose

Measure predictive performance.

Steps

1. Add a **Test and Score** widget.
2. Connect:
 - Select Columns → Data
 - Logistic Regression → Learner
3. Select **10-Fold Cross Validation**.
4. Run the evaluation.

Metrics to Record

- Classification Accuracy (CA)

- ROC-AUC
- F1 Score

Outcome

Performance metrics are generated automatically.

Step 6: Analyze Prediction Errors

Purpose

Identify correct and incorrect classifications.

Steps

1. Add a **Confusion Matrix** widget.
2. Connect Test and Score → Confusion Matrix.
3. Open the widget.
4. Review:
 - True Positives
 - True Negatives
 - False Positives
 - False Negatives

Outcome

Understand where the model makes mistakes.

Step 7: Generate Customer Predictions

Purpose

Apply the trained model to customer records.

Steps

1. Add a **Predictions** widget.
2. Connect:
 - Select Columns → Data
 - Logistic Regression → Predictors
3. Open the Predictions widget.

Outcome

Orange generates:

- Predicted class (0 or 1)
- Probability of churn

for each customer.

KPI Calculations

KPI 1: Churn Rate

Formula

Churn Rate (%) =

$(\text{Number of Customers with Exited} = 1 \div \text{Total Customers}) \times 100$

How to Obtain

1. Open Feature Statistics.
2. Find the count of customers where Exited = 1.
3. Divide by the total number of records.
4. Multiply by 100.

KPI 2: Retention Rate

Formula

Retention Rate (%) =

$(\text{Number of Customers with Exited} = 0 \div \text{Total Customers}) \times 100$

or

Retention Rate = 100 – Churn Rate

How to Obtain

Use the count of customers where Exited = 0 from Feature Statistics.

KPI 3: Model Accuracy

Formula

Accuracy (%) =

$(\text{Correct Predictions} \div \text{Total Predictions}) \times 100$

How to Obtain

1. Open Test and Score.
2. Locate the **CA (Classification Accuracy)** value.
3. Multiply by 100 if displayed as a decimal.

KPI 4: ROC-AUC

Purpose

Measures how well the model distinguishes churned customers from retained customers.

How to Obtain

1. Open Test and Score.
2. Record the **AUC** value.

Interpretation

- 0.90 – 1.00 → Excellent
- 0.80 – 0.89 → Very Good
- 0.70 – 0.79 → Acceptable
- 0.50 → Random Guessing

KPI Summary

KPI	Formula / Meaning	Orange Widgets
Churn Rate	$(\text{Exited} = 1 / \text{Total Customers}) \times 100$	Feature Statistics
Retention Rate	$(\text{Exited} = 0 / \text{Total Customers}) \times 100$	Feature Statistics
Accuracy	Correct Predictions / Total Predictions	Test and Score, Confusion Matrix
ROC-AUC	Model discrimination ability	Test and Score

Conclusion

This lab demonstrates how the CRISP-DM methodology can be applied to customer churn prediction. Using Orange Data Mining, students learn to import data, prepare features, build a machine learning model, evaluate performance, and interpret business-oriented KPIs. The resulting workflow provides valuable insights that can help organizations reduce customer churn and improve retention strategies.

LAB 3: Handling Missing Values, Duplicates & Noise

Introduction

High-quality data is essential for accurate healthcare analytics. However, real-world datasets often contain missing values, duplicate records, and abnormal values that can affect analysis results. Before applying machine learning or statistical techniques, the data must be cleaned and prepared.

This lab focuses on common data-cleaning techniques including duplicate removal, missing value treatment, and outlier handling. Students will use RapidMiner Studio and Python to transform raw healthcare data into a reliable dataset suitable for further analysis.

Problem Statement

A healthcare organization has collected large volumes of patient and billing information. Unfortunately, the dataset contains duplicate entries, incomplete records, and unrealistic numerical values that reduce data quality.

The objective is to build a data-cleaning workflow that identifies and corrects these issues while measuring the improvement using data quality KPIs.

Dataset Description

The lab uses a Healthcare Dataset stored in CSV format.

Important Attributes

Patient Identifiers

- patIndex
- patient_key

Demographic and Clinical Information

- age_cat
- sexN
- DIABETES
- ASTHMA
- HYPERTENSION

Financial and Utilization Data

- post_total_cost
- pre_ip_cost
- total_loss

Treatment Information

- drug_class

These attributes contain a mix of numerical and categorical data that require cleaning before analysis.

Tools Used

RapidMiner Studio

RapidMiner is a visual data analytics platform that allows users to create workflows using drag-and-drop operators.

Python

Python is used for advanced outlier detection and noise reduction that may not be easily performed using built-in RapidMiner operators.

Advantages

- Visual workflow design
- Easy data preprocessing
- Integration with Python scripts
- Suitable for large datasets

Learning Objectives

After completing this lab, students will be able to:

1. Import and inspect healthcare datasets.
2. Detect and remove duplicate records.
3. Handle missing values using imputation methods.
4. Reduce noise by treating outliers with Python.
5. Calculate data quality KPIs.

Procedure

Step 1: Import and Examine the Dataset

Purpose

Load the healthcare dataset and identify quality issues.

Steps

1. Add a **Read CSV** operator.
2. Select the Healthcare.csv file.
3. Connect it to the Results port.
4. Run the process.
5. Open the Statistics tab and record:
 - Total rows
 - Missing values
 - Attribute information

Step 2: Remove Duplicate Records

Purpose

Eliminate repeated patient records.

Steps

1. Insert a **Remove Duplicates** operator.
2. Connect it after Read CSV.
3. Configure the operator to check all attributes.
4. Execute the process.
5. Compare row counts before and after cleaning.

Outcome

Only unique records remain in the dataset.

Step 3: Handle Missing Values

Purpose

Replace incomplete data while preserving records.

Steps

1. Add a **Replace Missing Values** operator.
2. Connect it after Remove Duplicates.
3. Configure:
 - Numerical attributes → Average
 - Categorical attributes → Mode
4. Execute the workflow.
5. Verify that missing values have been removed.

Outcome

All required fields contain valid values.

Step 4: Filter Outliers Using Python

Purpose

Reduce the effect of unrealistic numerical values.

Steps

1. Add an **Execute Python** operator.
2. Connect it after Replace Missing Values.
3. Use Python to cap extreme values above the 99th percentile.
4. Execute the workflow.
5. Review updated statistics.

Outcome

Extreme outliers are reduced without deleting records.

KPI Calculations

KPI 1: Duplicate Removal Percentage

Formula

Duplicate Removal (%) =

$$((\text{Initial Rows} - \text{Final Rows}) \div \text{Initial Rows}) \times 100$$

Purpose

Measures how many duplicate records were removed.

KPI 2: Missing Value Reduction Percentage

Formula

Missing Reduction (%) =

$$((\text{Initial Missing Values} - \text{Final Missing Values}) \div \text{Initial Missing Values}) \times 100$$

Purpose

Measures the effectiveness of the imputation process.

KPI 3: Data Quality Score

Formula

Data Quality Score =

$$(\text{Completeness Rate} + \text{Uniqueness Rate}) \div 2$$

Where:

$$\text{Completeness Rate} = 100 - \text{Missing Data \%}$$

$$\text{Uniqueness Rate} = 100 - \text{Duplicate \%}$$

Purpose

Provides an overall indication of dataset quality.

KPI Summary

KPI	Formula	Tools Used
Missing Value Reduction %	$((\text{Initial Missing} - \text{Final Missing}) \div \text{Initial Missing}) \times 100$	Statistics, Replace Missing Values
Duplicate Removal %	$((\text{Initial Rows} - \text{Final Rows}) \div \text{Initial Rows}) \times 100$	Read CSV, Remove Duplicates

KPI	Formula	Tools Used
Data Quality Score	$(\text{Completeness} + \text{Uniqueness}) \div 2$	RapidMiner, Python

Conclusion

This lab demonstrates the importance of data cleaning in healthcare analytics. By removing duplicates, handling missing values, and reducing outliers, students create a cleaner and more reliable dataset suitable for machine learning and business analysis.

LAB 4: Normalization, Standardization & Discretization

Introduction

Machine learning models perform best when data is properly prepared. In many datasets, numerical attributes exist on different scales. For example, income values may reach hundreds of thousands, while age and credit score contain much smaller numbers. Such differences can bias machine learning algorithms and reduce prediction accuracy.

Data preprocessing solves this issue by transforming data into a consistent format. Common preprocessing techniques include normalization, standardization, and discretization. In this lab, students will apply these methods to a banking dataset to prepare it for future predictive modeling.

Problem Statement

A bank plans to develop a loan approval model using customer financial and demographic data. However, important attributes such as Income, Credit Score, and Loan Amount have very different numerical ranges.

To prevent larger values from dominating the learning process, the bank requires a preprocessing workflow that scales numerical data and groups customer ages into meaningful categories.

Dataset Description

This lab uses a Loan Prediction Dataset in CSV format.

Key Attributes

Numerical Attributes

- Age
- Income
- Credit_Score
- Loan_Amount

Categorical Attributes

- Employment_Status
- Loan_Approved

Data Characteristics

Attribute	Typical Range
------------------	----------------------

Age	18 – 69
-----	---------

Income	Up to 149,000
--------	---------------

Credit Score	300 – 850
--------------	-----------

Loan Amount	Several thousand to 50,000+
-------------	-----------------------------

These differences make preprocessing necessary before model training.

Tool Used

RapidMiner Studio

RapidMiner provides a visual environment for data transformation and preparation through drag-and-drop operators.

Benefits

- No programming required

- Easy workflow creation
- Built-in preprocessing operators
- Visual monitoring of data transformations

Learning Objectives

After completing this lab, students will be able to:

1. Import and analyze banking data.
2. Apply Min-Max normalization.
3. Perform Z-score standardization.
4. Convert numerical values into categories using discretization.
5. Evaluate preprocessing effectiveness using KPIs.

Procedure

Step 1: Load and Examine the Dataset

Purpose

Establish baseline statistics before preprocessing.

Steps

1. Add a **Read CSV** operator.
2. Import the loan dataset.
3. Connect it to the Results port.
4. Execute the process.
5. Record:
 - Mean
 - Standard Deviation
 - Variance
 - Minimum and Maximum values

Outcome

Identify attributes with large numerical ranges.

Step 2: Apply Min-Max Normalization

Purpose

Scale large financial values into a range between 0 and 1.

Steps

1. Add a **Normalize** operator.
2. Connect it after Read CSV.
3. Select:
 - Income
 - Loan_Amount
4. Choose:
 - Method = Range Transformation
 - Minimum = 0
 - Maximum = 1
5. Execute the process.

Outcome

Income and Loan Amount values are scaled between 0 and 1.

Step 3: Apply Z-Score Standardization

Purpose

Center Credit Score values around a mean of zero.

Steps

1. Add a second **Normalize** operator.
2. Connect it after the first Normalize operator.
3. Select:
 - Credit_Score
4. Choose:
 - Method = Z-Transformation
5. Execute.

Outcome

Credit scores become standardized values with positive and negative deviations from the mean.

Step 4: Discretize Age Values

Purpose

Convert continuous ages into grouped categories.

Steps

1. Add a **Discretize by Binning** operator.
2. Connect it after the second Normalize operator.
3. Select:
 - Age
4. Set:
 - Number of Bins = 3
5. Execute the process.

Outcome

Age values are converted into categorical age groups.

KPI Calculations

KPI 1: Variance Reduction

Formula

Variance Reduction =

Initial Variance – Final Variance

Purpose

Measures how effectively normalization reduces data spread.

Source

Compare Income variance before and after Min-Max normalization.

KPI 2: Distribution Improvement

Purpose

Measures how discretization improves data organization.

Method

Compare Age distribution:

Before Discretization:

- Many individual age values

After Discretization:

- Three grouped categories

Source

Review histograms and frequency distributions in RapidMiner statistics.

KPI Summary

KPI	Calculation	Operators Used
Variance Reduction	Initial Variance – Final Variance	Read CSV, Normalize
Distribution Improvement	Comparison of age distribution before and after binning	Discretize by Binning, Statistics

Conclusion

This lab demonstrates how preprocessing techniques improve data quality for machine learning. Through normalization, standardization, and discretization, raw banking data is transformed into a structured and balanced dataset suitable for predictive analytics and loan approval modeling.

LAB 5: Market Basket Analysis

Introduction

Retailers can gain valuable insights not only from what customers purchase but also from which products are frequently bought together. Market Basket Analysis (MBA) is a data mining technique used to discover relationships between products within customer transactions.

By identifying these associations, businesses can improve store layouts, create product bundles, and develop effective cross-selling strategies. In this lab, students will use the Apriori algorithm to find frequent itemsets and generate association rules based on transaction data.

Problem Statement

A supermarket wants to improve product placement and promotional campaigns. Although products are currently organized by category, management believes certain products are frequently purchased together and could be positioned strategically to increase sales.

The goal is to analyze historical transaction records, discover product associations, and identify combinations that can be used for cross-selling and marketing decisions.

Dataset Description

This lab uses the **Grocery Transactions Dataset** stored in CSV format.

Important Attributes

Attribute	Description
Member_number	Unique customer identifier
Date	Transaction date
itemDescription	Product purchased

Dataset Structure

Each row represents a single product purchased by a customer. Since Market Basket Analysis works on complete transactions, items purchased by the same customer on the same date must be grouped into a single basket.

Tools Used

Python

Python is used for data processing and association rule mining.

Libraries

Pandas

Used for:

- Data loading
- Data cleaning
- Basket creation
- Data transformation

Mlxtend

Used for:

- Apriori algorithm
- Frequent itemset generation
- Association rule extraction

Advantages

- Fast processing of transaction data
- Built-in implementation of Apriori
- Easy calculation of association metrics

Learning Objectives

After completing this lab, students will be able to:

1. Convert transaction records into basket format.
2. Perform one-hot encoding.
3. Calculate Support, Confidence, and Lift.
4. Generate frequent itemsets using Apriori.
5. Extract association rules.
6. Identify product combinations with cross-selling potential.

Association Rule Metrics

Before implementing Apriori, it is important to understand the key metrics used to evaluate product relationships.

Assume:

$A \rightarrow B$

means customers who buy Product A may also buy Product B.

Support

Support measures how frequently an item or item combination appears in all transactions.

Formula

$\text{Support}(A) =$

Number of Transactions Containing A $\div$ Total Transactions

Example

If milk appears in 2,363 transactions out of 14,963:

$$\text{Support} = 2363 \div 14963$$

$$\text{Support} = 0.1579$$

or

15.79%

Confidence

Confidence measures the probability of purchasing B when A has already been purchased.

Formula

$$\text{Confidence}(A \rightarrow B) =$$

$$\text{Support}(A \cup B) \div \text{Support}(A)$$

Example

If:

- Rolls/Buns appears in 1,646 transactions
- Rolls/Buns and Milk appear together in 222 transactions

Confidence =

$$222 \div 1646$$

$$= 0.1348$$

or

$$13.48\%$$

Lift

Lift measures the strength of the association between two products.

Formula

$$\text{Lift}(A \rightarrow B) =$$

$$\text{Confidence}(A \rightarrow B) \div \text{Support}(B)$$

Interpretation

Lift Value Meaning

> 1 Positive association

= 1 No association

< 1 Negative association

Higher lift values indicate stronger cross-selling opportunities.

Procedure

Step 1: Load the Dataset

Purpose

Import transaction records into Python.

Code

```
import pandas as pd

df = pd.read_csv("Groceries_dataset.csv")

print(df.head())
```

Verification

Check that the dataset loads successfully and contains:

- Member_number
- Date
- itemDescription

```
[1]: import pandas as pd
      df = pd.read_csv(r"C:\Users\Rihab\Downloads\archive (9)\Groceries_dataset.csv")
      print(df.head())
```

	Member_number	Date	itemDescription
0	1808	21-07-2015	tropical fruit
1	2552	05-01-2015	whole milk
2	2300	19-09-2015	pip fruit
3	1187	12-12-2015	other vegetables
4	3037	01-02-2015	whole milk

Step 2: Create Transaction IDs

Purpose

Combine customer ID and transaction date to form unique baskets.

Code

```
df["Transaction_ID"] = (  
  
    df["Member_number"].astype(str)  
  
    + " "  
  
    + df["Date"].astype(str)  
  
)
```

Verification

Each shopping visit should have a unique transaction ID.

```
df["Transaction_ID"] = (  
    df["Member_number"].astype(str)  
    + " "  
    + df["Date"].astype(str)  
)
```

Step 3: Create Basket Matrix

Purpose

Convert transaction records into basket format.

Code

```
basket = (  
  
    df.groupby(  
  
        ["Transaction_ID", "itemDescription"]  
  
    )["itemDescription"]  
  
    .count()  
  
    .unstack()  
  
    .fillna(0)  
  
)
```

Verification

Rows represent transactions.

Columns represent products.

```
basket = (  
    df.groupby(  
        ["Transaction_ID", "itemDescription"]  
    )["itemDescription"]  
    .count()  
    .unstack()  
    .fillna(0)  
)
```

Step 4: Perform One-Hot Encoding

Purpose

Convert quantities into binary values.

Code

```
def encode_units(x):  
  
    return 1 if x >= 1 else 0  
  
basket_sets = basket.applymap(encode_units)  
  
N = len(basket_sets)  
  
print("Total Transactions:", N)
```

Verification

All values should be:

- 1 = Product Present
- 0 = Product Absent

```
def encode_units(x):  
    return 1 if x >= 1 else 0  
basket_sets = basket.applymap(encode_units)  
N = len(basket_sets)  
print("Total Transactions:", N)
```

C:\Users\Rihab\AppData\Local\Temp\ipykernel_8924\2468094540.py:3: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.

```
    basket_sets = basket.applymap(encode_units)  
Total Transactions: 14963
```

Step 5: Calculate Support, Confidence, and Lift

Purpose

Manually calculate association metrics.

Example Rule

rolls/buns → whole milk

Code

```
item_A = "rolls/buns"
```

```
item_B = "whole milk"
```

```
freq_A = basket_sets[item_A].sum()
```

```
freq_B = basket_sets[item_B].sum()
```

```
freq_AB = (  
    (basket_sets[item_A] == 1) &  
    (basket_sets[item_B] == 1)  
).sum()
```

```
support_A = freq_A / N
```

$\text{support_B} = \text{freq_B} / N$

$\text{support_AB} = \text{freq_AB} / N$

$\text{confidence} = \text{support_AB} / \text{support_A}$

$\text{lift} = \text{confidence} / \text{support_B}$

`print("Support:", support_AB)`

`print("Confidence:", confidence)`

`print("Lift:", lift)`

Verification

The calculated values should be valid decimal values between 0 and 1 (except Lift, which may exceed 1).

```
: item_A = "rolls/buns"
  item_B = "whole milk"
  freq_A = basket_sets[item_A].sum()
  freq_B = basket_sets[item_B].sum()

  freq_AB = (
      (basket_sets[item_A] == 1) &
      (basket_sets[item_B] == 1)
  ).sum()

  support_A = freq_A / N
  support_B = freq_B / N
  support_AB = freq_AB / N

  confidence = support_AB / support_A
  lift = confidence / support_B

  print("Support:", support_AB)
  print("Confidence:", confidence)
  print("Lift:", lift)
```

```
Support: 0.013967787208447505
Confidence: 0.12697448359659783
Lift: 0.8040284376030019
```

Step 6: Generate Frequent Itemsets

Purpose

Identify products that frequently appear together.

Code

```
from mlxtend.frequent_patterns import apriori

frequent_itemsets = apriori(

    basket_sets,

    min_support=0.01,

    use_colnames=True

)

print(frequent_itemsets.head())
```

Verification

Frequent itemsets should be displayed with support values.

Step 7: Generate Association Rules

Purpose

Identify strong product relationships.

Code

```
from mlxtend.frequent_patterns import association_rules

rules = association_rules(

    frequent_itemsets,

    metric="lift",

    min_threshold=1.0

)
```

Verification

Rules with Lift greater than 1 should be generated.

Step 8: Rank Cross-Selling Opportunities

Purpose

Identify the strongest product combinations.

Code

```
best_rules = rules.sort_values(

    by="lift",
```

```
ascending=False
```

```
)
```

```
print(best_rules.head(10))
```

Verification

Rules with the highest Lift values appear at the top.

Results and Discussion

After applying Market Basket Analysis, several purchasing patterns can be identified.

Frequent Itemsets

Products such as:

- Whole Milk
- Rolls/Buns
- Other Vegetables

typically show high support because they are commonly purchased items.

High Lift Rules

Products with Lift greater than 1 indicate meaningful associations and potential cross-selling opportunities.

Business Applications

The discovered rules can be used for:

- Shelf placement optimization
- Product bundling
- Promotional campaigns
- Recommendation systems

KPI Summary

KPI	Calculation	Python Tools Used
Frequent Itemsets	$\text{Support} \geq \text{Minimum Threshold}$	Apriori
High Lift Rules	$\text{Lift} > 1$	Association Rules
Cross-Sell Potential Ranking Rules	by Lift and Confidence	Pandas Sorting

Conclusion

This lab introduced Market Basket Analysis using Python and the Apriori algorithm. By transforming transaction records into basket data, calculating Support, Confidence, and Lift, and generating association rules, students can uncover hidden product relationships

and identify valuable cross-selling opportunities. These insights can help retailers improve product placement, increase sales, and enhance customer purchasing experiences.

LAB 6: Apriori Algorithm

Introduction

Organizations often analyze customer transactions to understand purchasing behavior and improve marketing strategies. One of the most widely used techniques for discovering relationships between products or services is the **Apriori Algorithm**.

Apriori is an association rule mining algorithm that identifies items frequently occurring together within transactions. In the banking sector, it can be used to analyze merchant spending patterns and discover relationships between vendors frequently used by the same customers.

In this lab, students will transform banking transaction records into transaction baskets and apply the Apriori algorithm to generate association rules based on Support, Confidence, and Lift.

Problem Statement

A bank wants to improve its rewards and promotional programs by understanding customer spending habits.

Although transaction records are available, the bank cannot easily identify which merchants are commonly used together by the same customers. The goal is to build an Apriori-based analysis workflow that discovers merchant associations and helps management design targeted offers and partnerships.

Dataset Description

The lab uses the **bank_transactions_data_2.csv** dataset.

Important Attributes

Attribute	Description
AccountID	Unique customer account number
MerchantID	Merchant or vendor identifier
TransactionType	Debit, Credit, etc.
Channel	ATM, Online, POS, etc.

Name	Date modified	Type	Size
Today			
 bank_transactions_data_2	16/06/2026 7:06 am	Microsoft Excel Co...	325 KB

Dataset Structure

Each row represents a single banking transaction.

For Apriori analysis, transactions belonging to the same AccountID must be grouped together to create customer baskets.

Tools Used

KNIME Analytics Platform

Used for:

- Data loading
- Aggregation
- Data preparation
- Workflow design

Python Script Node

Used for:

- One-hot encoding
- Apriori algorithm execution
- Association rule generation

Python Libraries

- pandas
- mlxtend

Learning Objectives

After completing this lab, students will be able to:

1. Import banking transaction data into KNIME.
2. Group transactions into customer baskets.
3. Create a binary transaction matrix.
4. Generate frequent itemsets using Apriori.
5. Extract association rules.
6. Evaluate the effect of different support thresholds.

Procedure

Step 1: Import the Dataset

Purpose

Load banking transaction records into KNIME.

Steps

1. Add a **CSV Reader** node.
2. Select:

bank_transactions_data_2.csv

3. Execute the node.
4. Open the output table.

Verification

Confirm that:

- AccountID exists
- MerchantID exists
- Each row represents a transaction

Rows: 2512 | Columns: 16

Table  Statistics 



RowID	Transacti... T String	AccountID T String	Transacti... .00 Number (...)	Transacti... T String	Transacti... T String	Location T String
Row1	TX000002	AC00455	376.24	27/06/2023 16:	Debit	Houston
Row2	TX000003	AC00019	126.29	10/07/2023 18:	Debit	Mesa
Row3	TX000004	AC00070	184.5	05/05/2023 16:	Debit	Raleigh
Row4	TX000005	AC00411	13.45	16/10/2023 17:	Credit	Atlanta
Row5	TX000006	AC00393	92.15	03/04/2023 17:	Debit	Oklahoma City

Step 2: Create Customer Baskets

Purpose

Combine all merchant transactions for each customer.

Steps

1. Add a **GroupBy** node.
2. Connect it to CSV Reader.
3. Configure:

Groups Tab

Select:

- AccountID

Manual Aggregation Tab

Select:

- MerchantID

Aggregation Method:

- List
- or

- Concatenate

4. Execute the node.

Verification

Each row should represent one customer account containing multiple merchants.

Rows: 495 | Table | Statistics

#	RowID	AccountID String	List(Merc... Collection...
1	Row0	AC00001	[M034,M003]
2	Row1	AC00002	[M040,M087,M003]
3	Row2	AC00003	[M089,M032,M003]
4	Row3	AC00004	[M064,M061,M003]
5	Row4	AC00005	[M036,M002,M003]
6	Row5	AC00006	[M036,M087,M003]

Step 3: Create Binary Matrix

Purpose

Convert merchant lists into a one-hot encoded matrix.

Steps

1. Add a **Python Script** node.
2. Connect it after GroupBy.
3. Use Python to create binary merchant columns.

Verification

Output should resemble:

AccountID M001 M002 M003

AC001 1 0 1

AC002 0 1 1

Step 4: Run Apriori Algorithm

Purpose

Generate frequent merchant combinations.

Steps

1. Import Apriori functions.
2. Set a minimum support threshold.
3. Generate frequent itemsets.

Example

```
frequent_itemsets = apriori(  
  
    basket,  
  
    min_support=0.05,  
  
    use_colnames=True  
)
```

Verification

Output should contain:

- Itemsets
- Support values

Step 5: Generate Association Rules

Purpose

Identify merchant relationships.

Steps

1. Use `association_rules()`.
2. Set confidence or lift threshold.
3. Execute.

Example

```
rules = association_rules(  
  
    frequent_itemsets,  
  
    metric="confidence",  
  
    min_threshold=0.5
```

)

Verification

Output should contain:

- Antecedents
- Consequents
- Support
- Confidence
- Lift

Step 6: Analyze Support Thresholds

Purpose

Observe how support affects the number of generated rules.

Steps

Run Apriori multiple times using:

- 0.01
- 0.03
- 0.05

- 0.10

Record:

- Frequent itemsets
- Generated rules

Observation

Higher support values generally produce fewer rules.

KPI Calculations

KPI 1: Number of Rules

Formula

Number of Rules =

Total rows in the association rules output

Purpose

Measures the amount of actionable knowledge generated by Apriori.

KPI 2: Support Threshold Impact

Formula

Support Impact =

Rules at Threshold X – Rules at Threshold Y

Purpose

Measures how changing support affects rule generation.

Interpretation

- Lower support → More rules
- Higher support → Fewer but stronger rules

KPI Summary

KPI	Calculation	Tools Used
Number of Rules	Count of generated association rules	Python Script
Support Threshold Impact	Difference in rules across support levels	Apriori Results

Conclusion

This lab demonstrated how the Apriori algorithm can be applied to banking transaction data to discover merchant associations. By transforming raw transaction logs into customer baskets and generating association rules, banks can identify spending patterns, improve promotional campaigns, and develop more effective customer loyalty programs.

LAB 7: FP-Tree Construction & FP-Growth

INTRODUCTION

Frequent Pattern Growth (FP-Growth) is an advanced algorithm used in market basket analysis to efficiently extract frequent itemsets from large transactional datasets. Unlike the Apriori algorithm, which repeatedly scans the dataset and generates a large number of candidate itemsets, FP-Growth compresses the dataset into a compact data structure called the **FP-Tree (Frequent Pattern Tree)**.

This tree-based structure allows the algorithm to mine frequent patterns directly without candidate generation, making it significantly faster and more scalable for large datasets.

PROBLEM STATEMENT

A global retail enterprise is struggling to analyze massive transactional data using the Apriori algorithm due to memory and performance limitations. The system frequently slows down or crashes when processing large datasets.

To overcome this issue, the organization requires a more efficient approach using FP-Growth, which can extract frequent patterns and association rules in significantly less time while maintaining accuracy. This will enable real-time decision-making for cross-selling and product bundling strategies.

DATASET DESCRIPTION

The dataset (new_retail_data.csv) contains global retail transactions.

Key Attributes:

- **Customer_ID** → Unique identifier for customer
- **Date** → Transaction date
- **products** → Purchased item (e.g., Laptop, Shoes, Cookies)

Each row represents a single product purchase. We must group these into **basket transactions** before applying FP-Growth.

TOOLS USED

- Python
- Pandas
- mlxtend (for FP-Growth and association rules)
- time module (for execution benchmarking)
- Orange Data Mining (visual validation)

PRACTICAL OBJECTIVES

- Convert raw transactional data into basket format
- Apply FP-Growth algorithm to extract frequent itemsets
- Generate association rules using lift and confidence
- Compare FP-Growth performance with Apriori
- Measure execution time and rule count

STEP-BY-STEP IMPLEMENTATION

STEP 1: DATA PREPARATION

```
import pandas as pd

df = pd.read_csv(r"C:\Users\Rihab\Downloads\archive (11)\Retail_Transactions_Dataset.csv")

# Create Basket ID
df['Basket_ID'] = df['Customer_ID'].astype(str) + "_" + df['Date'].astype(str)

# Group into basket format
basket = df.groupby(['Basket_ID', 'products'])['products'] \
          .count().unstack().fillna(0)

# Convert to 1/0 encoding
basket_sets = basket.applymap(lambda x: 1 if x >= 1 else 0)

print(f"Total Unique Baskets: {len(basket_sets)}")
```

STEP 2: IMPORT LIBRARIES

```
; from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules
import time
```

STEP 3: APRIORI (BASELINE)

```
print("\n--- Running Apriori ---")

start_time_apriori = time.time()

frequent_itemsets_ap = apriori(
    basket_sets,
    min_support=0.01,
    use_colnames=True
)

rules_ap = association_rules(
    frequent_itemsets_ap,
    metric="lift",
    min_threshold=1.0
)

end_time_apriori = time.time()

apriori_execution_time = end_time_apriori - start_time_apriori

print("Apriori Execution Time:", apriori_execution_time)
print("Apriori Rule Count:", len(rules_ap))
```

STEP 4: FP-GROWTH IMPLEMENTATION

```
print("\n--- Running FP-Growth ---")

start_time_fp = time.time()

frequent_itemsets_fp = fpgrowth(
    basket_sets,
    min_support=0.01,
    use_colnames=True
)

rules_fp = association_rules(
    frequent_itemsets_fp,
    metric="lift",
    min_threshold=1.0
)

end_time_fp = time.time()

fp_execution_time = end_time_fp - start_time_fp

print("FP-Growth Execution Time:", fp_execution_time)
print("FP-Growth Rule Count:", len(rules_fp))
```

RESULTS AND OBSERVATIONS

- FP-Growth and Apriori generate the **same association rules**
- FP-Growth is significantly **faster**
- Apriori slows down due to repeated dataset scanning
- FP-Growth builds an FP-Tree and avoids candidate generation

Key Insight:

- ✓ Same accuracy
- ✓ Same rule count
- ✓ FP-Growth is much faster

KEY PERFORMANCE INDICATORS (KPIs)

EXECUTION TIME

Execution Time = End Time - Start Time

- Apriori Execution Time → apriori_execution_time
- FP-Growth Execution Time → fp_execution_time

RULE COUNT

len(rules_ap)

len(rules_fp)

KPI SUMMARY TABLE

KPI	Method	Tool
Execution Time	Time difference between start and end Python time module	
Rule Count	Number of generated rules	len() function
Frequent Itemsets	Apriori / FP-Growth output	mlxtend
Association Rules	Lift-based filtering	association_rules

CONCLUSION

FP-Growth is a highly efficient improvement over the Apriori algorithm. It eliminates the costly candidate generation process by using an FP-Tree structure, allowing faster mining of frequent patterns. In large-scale retail datasets, FP-Growth provides the same analytical accuracy as Apriori but with significantly reduced execution time, making it suitable for real-time and big data applications.

LAB 8: Python Syntax & Basics

INTRODUCTION

Until now, data mining workflows have largely depended on visual tools such as KNIME, Orange, and RapidMiner. These platforms are useful for rapid prototyping and building pipelines without coding.

However, real-world data science requires deeper control, automation, and flexibility—something that is best achieved through programming. In this lab, we transition from visual workflows to **pure Python programming**.

The focus of this lab is to understand:

- Python **lists and dictionaries** for structured data storage
- Basic data manipulation without external libraries
- Introduction to **Object-Oriented Programming (OOP)**
- Designing a reusable **data processing class**

Instead of writing repeated scripts every week, we will build a reusable system that automates cleaning and processing tasks.

PROBLEM STATEMENT

The university's academic affairs office receives weekly student performance reports containing grades and attendance records. These datasets are often inconsistent, including:

- Missing test scores
- Inconsistent name capitalization
- Irregular formatting

Currently, staff manually clean this data using spreadsheets, which is time-consuming and inefficient.

The goal of this lab is to design a **reusable Python-based data processing module** that:

- Automatically cleans and standardizes data
- Handles missing values
- Normalizes text formatting
- Computes basic performance metrics

This will allow future datasets to be processed with minimal code.

3. DATASET DESCRIPTION

For simplicity, we simulate the dataset directly in Python using native data structures instead of importing external files.

Each record contains:

- **Student_ID** → Unique identifier for each student
- **Name** → Student name (may have inconsistent capitalization)

- **Attendance** → Attendance percentage
- **Math_Score** → Marks obtained in Mathematics (0–100, may contain missing values)
- **Science_Score** → Marks obtained in Science (0–100, may contain missing values)

Data Structure Used:

- Each student record → **Dictionary**
- Collection of records → **List of dictionaries**

TOOLS USED

This lab uses **pure Python (standard library only)**.

No external libraries such as pandas or NumPy are used to ensure:

- Strong understanding of core Python logic
- Better grasp of loops, conditions, and data structures
- Hands-on experience with OOP principles

You can run the code in:

- Jupyter Notebook
- Any Python IDE (PyCharm, VS Code, etc.)

PRACTICAL OBJECTIVES

By the end of this lab, students will be able to:

- Store and manipulate structured data using Python lists and dictionaries
- Design a reusable Python class for data processing
- Handle missing values using logical imputation techniques
- Normalize and clean text data programmatically
- Evaluate code efficiency and reusability

STEP-BY-STEP PROCEDURE & PYTHON IMPLEMENTATION

STEP 1: SIMULATING RAW DATA (LISTS & DICTIONARIES)

We begin by creating a small dataset manually using dictionaries (records) inside a list.

Each dictionary represents one student record containing scores, attendance, and personal details.

```
[1]: # Simulated dataset using List of dictionaries
students = [
    {"Student_ID": "S001", "Name": "aLIce", "Attendance": 85, "Math_Score": 78, "Science_Score": 82},
    {"Student_ID": "S002", "Name": "bob", "Attendance": 90, "Math_Score": None, "Science_Score": 88},
    {"Student_ID": "S003", "Name": "CHARLIE", "Attendance": 70, "Math_Score": 65, "Science_Score": None},
    {"Student_ID": "S004", "Name": "dAvId", "Attendance": 95, "Math_Score": 92, "Science_Score": 89}
]

students

[1]: [{'Student_ID': 'S001',
      'Name': 'aLIce',
      'Attendance': 85,
      'Math_Score': 78,
      'Science_Score': 82},
      {'Student_ID': 'S002',
      'Name': 'bob',
      'Attendance': 90,
      'Math_Score': None,
      'Science_Score': 88},
      {'Student_ID': 'S003',
      'Name': 'CHARLIE',
      'Attendance': 70,
      'Math_Score': 65,
      'Science_Score': None},
      {'Student_ID': 'S004',
      'Name': 'dAvId',
      'Attendance': 95,
      'Math_Score': 92,
      'Science_Score': 89}]
```

STEP 2: DESIGNING THE DATA PROCESSOR CLASS

We define a class called DataProcessor.

A class acts as a **blueprint for processing data**, allowing us to:

- Reuse logic for multiple datasets
- Avoid rewriting code
- Encapsulate cleaning functions inside a single structure

```

2]:
import time

class DataProcessor:
    def __init__(self, data):
        self.data = data

    def clean_names(self):
        for student in self.data:
            student["Name"] = student["Name"].strip().title()
        return self.data

    def handle_missing(self):
        math_vals = [s["Math_Score"] for s in self.data if s["Math_Score"] is not None]
        sci_vals = [s["Science_Score"] for s in self.data if s["Science_Score"] is not None]

        math_mean = sum(math_vals) / len(math_vals)
        sci_mean = sum(sci_vals) / len(sci_vals)

        for student in self.data:
            if student["Math_Score"] is None:
                student["Math_Score"] = math_mean
            if student["Science_Score"] is None:
                student["Science_Score"] = sci_mean

        return self.data

    def run_pipeline(self):
        self.clean_names()
        self.handle_missing()
        return self.data

```

STEP 3: BUILDING PREPROCESSING METHODS

Inside the class, we implement key cleaning functions:

- **Name Cleaning**
 - Standardizes capitalization (e.g., "aLIce" → "Alice")
- **Missing Value Handling**
 - Replaces missing scores (None) with mean or default values
- **Data Validation**

- Ensures all numeric fields are within valid ranges

These methods form the core preprocessing pipeline.

STEP 4: EXECUTING THE PIPELINE

Once the class is built, processing new datasets becomes extremely simple:

- Create an object of DataProcessor
- Pass dataset into it
- Call a single function like run_pipeline()

This demonstrates how OOP improves automation and reduces code complexity.

```
3]: processor = DataProcessor(students)

start = time.time()
cleaned_data = processor.run_pipeline()
end = time.time()

cleaned_data
```

```
3]: [{'Student_ID': 'S001',
      'Name': 'Alice',
      'Attendance': 85,
      'Math_Score': 78,
      'Science_Score': 82},
      {'Student_ID': 'S002',
      'Name': 'Bob',
      'Attendance': 90,
      'Math_Score': 78.33333333333333,
      'Science_Score': 88},
      {'Student_ID': 'S003',
      'Name': 'Charlie',
      'Attendance': 70,
      'Math_Score': 65,
      'Science_Score': 86.33333333333333},
      {'Student_ID': 'S004',
      'Name': 'David',
      'Attendance': 95,
      'Math_Score': 92,
      'Science_Score': 89}]
```

KEY PERFORMANCE INDICATORS (KPIs)

Since this lab focuses on software design rather than machine learning, we evaluate performance using **engineering metrics**.

CODE REUSABILITY

- **Objective:** Measure how efficiently the code can be reused for new datasets
- **Method:**
 - Initial implementation requires ~20 lines of code
 - New datasets require only:
 - Object creation
 - One function call

Example:

```
processor = DataProcessor(new_data)
```

```
processor.run_pipeline()
```

This shows a major improvement in reusability and maintainability.

CODE EFFICIENCY (EXECUTION TIME)

- **Objective:** Measure runtime performance of the pipeline

- **Method:**

- Use Python's time module
- Record start and end time of run_pipeline()

Formula:

Execution Time = End Time - Start Time

```
[4]: execution_time = end - start  
     execution_time
```

```
[4]: 0.0
```

Since the dataset is small and uses native Python structures, execution is extremely fast (O(N) complexity).

KPI SUMMARY TABLE

KPI Name	Calculation Method	Tools Used
Code Efficiency	End Time – Start Time	Python time module
Reusability	Lines of code required for new datasets	Custom class (DataProcessor)

CONCLUSION

This lab demonstrates the shift from procedural scripting to **object-oriented programming in Python**.

By using lists, dictionaries, and classes, we successfully built a reusable data processing system that:

- Automates data cleaning
- Handles missing values
- Standardizes formatting
- Reduces manual workload significantly

LAB 9: Decision Tree Classifier

INTRODUCTION

Telemarketing campaigns often perform poorly because customers are contacted randomly, leading to low conversion rates and wasted resources. For banks promoting term deposits, this approach is inefficient and costly.

To solve this, machine learning—specifically the **Decision Tree algorithm**—is used to identify customers who are more likely to subscribe. A Decision Tree is a simple yet powerful model that classifies data by applying a sequence of logical if–else rules, making its decisions easy to interpret compared to complex “black-box” models.

In this lab, we implement a full classification workflow in **Jupyter Notebook using Python and scikit-learn** to predict customer subscription behavior.

PROBLEM STATEMENT

A bank wants to improve its marketing strategy for term deposits by reducing unnecessary calls. Currently, agents contact customers without any predictive guidance, resulting in low efficiency.

The goal is to build a classification model that analyzes historical marketing data and predicts whether a customer will subscribe to the term deposit. This will help the bank focus only on high-probability customers and improve campaign performance.

DATASET DESCRIPTION

The dataset used is **bank.csv**, containing records from previous marketing campaigns.

Feature Groups:

- **Demographic Information:** age, job, marital status, education
- **Financial Indicators:** balance, housing loan, personal loan, default status
- **Campaign Interaction Data:** number of contacts (campaign), previous contact outcomes
- **Target Variable:**
 - deposit → indicates subscription outcome (yes or no)

This dataset is used to train a model that learns patterns behind customer decisions.

TOOLS & TECHNOLOGIES

This lab is implemented in:

- **Jupyter Notebook**
- **Python (Pandas, NumPy)**
- **Scikit-learn (Decision Tree Classifier)**

Python is used for full pipeline control including data preprocessing, model training, prediction, and evaluation. This provides more flexibility compared to visual tools.

OBJECTIVES

After completing this lab, students will be able to:

- Clean dataset and remove irrelevant or leakage-prone features
- Split data into training and testing sets using stratified sampling
- Train a Decision Tree classifier using scikit-learn
- Generate predictions on unseen data
- Evaluate model performance using Accuracy, Precision, Recall, and F1-score
- Interpret results for business decision-making

METHODOLOGY (STEP-BY-STEP IN JUPYTER)

STEP 1: DATA LOADING & CLEANING

- Load bank.csv using pandas
- Remove the feature duration to prevent data leakage (since it is only known after the call ends)

```
[1]: import pandas as pd

df = pd.read_csv(r"C:\Users\Rihab\Downloads\archive (12)\Churn_Modelling.csv")
df.head()
```

```
[1]:
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1

STEP 2: DATA PREPARATION & ENCODING

- Convert categorical variables into numeric form using encoding techniques
- Separate features (X) and target (y)

```
:
from sklearn.preprocessing import LabelEncoder

df_encoded = df.copy()
le = LabelEncoder()

for col in df_encoded.select_dtypes(include='object').columns:
    df_encoded[col] = le.fit_transform(df_encoded[col])

df_encoded.head()
```

```
:
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Balance	NumOfProducts	HasCrCard	IsActive
0	1	15634602	1115	619	0	0	42	0.00	1	1	
1	2	15647311	1177	608	2	0	41	83807.86	1	0	
2	3	15619304	2040	502	0	0	42	159660.80	3	1	
3	4	15701354	289	699	0	0	39	0.00	2	0	
4	5	15737888	1822	850	2	0	43	125510.82	1	1	

STEP 3: TRAIN-TEST SPLIT (STRATIFIED)

- Split dataset into 80% training and 20% testing
- Maintain class balance using stratification

```
from sklearn.model_selection import train_test_split

X = df_encoded.drop("Balance", axis=1)
y = df_encoded["Balance"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

STEP 4: MODEL TRAINING (DECISION TREE)

- Train a Decision Tree Classifier using Gini Index

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X_train, y_train)
```

STEP 5: PREDICTION

```
y_pred = model.predict(X_test)
y_pred[:10]
```

STEP 6: MODEL EVALUATION

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

KEY PERFORMANCE INDICATORS (KPI ANALYSIS)

Accuracy

Measures overall correctness of predictions (both yes and no cases).

Useful for general performance but can be misleading in imbalanced datasets.

Precision

Shows how many predicted “buyers” were actually correct.

High precision means fewer wasted marketing calls.

Recall

Measures how many actual buyers were correctly identified.

High recall ensures potential customers are not missed.

F1-Score

A balanced measure combining precision and recall.

Useful when both false positives and false negatives matter.

KPI SUMMARY TABLE

Metric	Meaning	Python Function
Accuracy	Overall correctness	<code>accuracy_score()</code>
Precision	Correct positive predictions	<code>precision_score()</code>
Recall	Coverage of actual positives	<code>recall_score()</code>
F1 Score	Balanced performance measure	<code>f1_score()</code>

CONCLUSION

This lab demonstrated the implementation of a Decision Tree classifier using **Jupyter Notebook and Python** for telemarketing prediction. The workflow included data

preprocessing, leakage removal, stratified splitting, model training, and performance evaluation.

The final model helps identify high-probability customers, allowing banks to optimize marketing efforts and reduce unnecessary operational costs through data-driven decision-making.

LAB 10: Naïve Bayes & KNN

INTRODUCTION

Spam emails reduce communication efficiency and security. Machine learning helps automatically classify emails as **spam (1)** or **ham (0)** using text patterns.

This lab compares **Naïve Bayes** and **K-Nearest Neighbors (KNN)** for spam detection.

PROBLEM STATEMENT

Build a text classification model that predicts whether an email is spam. Compare Naïve Bayes and KNN based on accuracy and false positive rate (FPR), where minimizing misclassified legitimate emails is critical.

DATASET

- **text** → email content
- **spam** → target label (1 = spam, 0 = not spam)

IMPORT & LOAD DATA

```
[1]: import pandas as pd

df = pd.read_csv(r"C:\Users\Rihab\Downloads\archive (13).zip")
df.head()
```

```
[1]:
```

	crl.tot	dollar	bang	money	n000	make	yesno
0	278	0.000	0.778	0.00	0.00	0.00	y
1	1028	0.180	0.372	0.43	0.43	0.21	y
2	2259	0.184	0.276	0.06	1.16	0.06	y
3	191	0.000	0.137	0.00	0.00	0.00	y
4	191	0.000	0.135	0.00	0.00	0.00	y

TEXT PREPROCESSING

Convert text into numeric features using TF-IDF.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(stop_words="english", lowercase=True)
X = tfidf.fit_transform(df["text"])
y = df["spam"]
```

TRAIN-TEST SPLIT

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
```

NAÏVE BAYES MODEL

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
```

K-NEAREST NEIGHBORS (KNN) MODEL

```
from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)

knn_pred = knn_model.predict(X_test)
```

EVALUATION METRICS

```
from sklearn.metrics import accuracy_score, confusion_matrix

def evaluate(y_test, y_pred):
    acc = accuracy_score(y_test, y_pred)
    tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
    fpr = fp / (fp + tn)
    return acc, fpr
```

Naïve Bayes Results

```
nb_acc, nb_fpr = evaluate(y_test, nb_pred)
print("Naïve Bayes Accuracy:", nb_acc)
print("Naïve Bayes FPR:", nb_fpr)
```

KNN Results

```
knn_acc, knn_fpr = evaluate(y_test, knn_pred)
print("KNN Accuracy:", knn_acc)
print("KNN FPR:", knn_fpr)
```

INTERPRETATION

- **Accuracy** → overall correctness
- **False Positive Rate (FPR)** → most important (wrongly marking real emails as spam is critical error)
- Best model = **lower FPR + acceptable accuracy**

KPI SUMMARY

Metric	Formula	Purpose
--------	---------	---------

Accuracy	Correct / Total	Overall performance
----------	-----------------	---------------------

Metric	Formula	Purpose
FPR	$FP / (FP + TN)$	Spam misclassification risk

CONCLUSION

Naïve Bayes is fast and strong for text classification, while KNN depends on similarity and can reduce or increase false positives depending on data. Final model selection prioritizes **lowest FPR for real-world email safety**.

LAB 11: Support Vector Machine (SVM)

INTRODUCTION

Credit card fraud detection is a **binary classification problem** (Fraud vs Legit).

We use **Support Vector Machine (SVM)** to find an optimal separating hyperplane.

- Linear SVM → simple separation
- RBF SVM → handles complex, non-linear fraud patterns
- Goal → maximize **Recall + Precision on imbalanced data**

PROBLEM STATEMENT

Financial institutions face:

- High false positives (blocking valid users)
- Missed fraud cases (financial loss)

Need a model that:

- Detects fraud accurately
- Handles **class imbalance**
- Maintains high **Recall without losing Precision**

DATASET DESCRIPTION

File: data2.csv

Features:

- Profession (categorical)
- Income (numerical)
- Fraud (target: 0 = legit, 1 = fraud)

Drop irrelevant columns:

- Credit_card_number
- Expiry
- Security_code

IMPORT LIBRARIES

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.svm import SVC

from sklearn.metrics import precision_score, recall_score, roc_auc_score
```

LOAD + CLEAN DATA

```
df = pd.read_csv("data2.csv")  
  
df = df.drop(columns=["Credit_card_number", "Expiry", "Security_code"])  
df.head()
```

ENCODING CATEGORICAL DATA

```
df = pd.get_dummies(df, columns=["Profession"], drop_first=True)
```

SPLIT FEATURES AND TARGET

```
|: X = df.drop("Fraud", axis=1)  
   y = df["Fraud"]
```

NORMALIZATION

```
scaler = MinMaxScaler()  
X_scaled = scaler.fit_transform(X)
```

TRAIN-TEST SPLIT (STRATIFIED)

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y,  
    test_size=0.2,  
    stratify=y,  
    random_state=42  
)
```

TRAIN SVM MODELS

Linear Kernel

```
svm_linear = SVC(kernel="linear")  
svm_linear.fit(X_train, y_train)
```

RBF Kernel

```
svm_rbf = SVC(kernel="rbf")  
svm_rbf.fit(X_train, y_train)
```

PREDICTIONS

```
y_pred_linear = svm_linear.predict(X_test)  
y_pred_rbf = svm_rbf.predict(X_test)
```

EVALUATION METRICS

Linear SVM Results

```
print("LINEAR SVM")
print("Precision:", precision_score(y_test, y_pred_linear))
print("Recall:", recall_score(y_test, y_pred_linear))
print("ROC-AUC:", roc_auc_score(y_test, y_pred_linear))
```

I TNFAR SVM

KPI FORMULAS

Recall (Fraud Detection Rate)

$$TP / (TP + FN)$$

Precision (Correct Alerts)

$$TP / (TP + FP)$$

CONCLUSION

SVM effectively separates fraud from legitimate transactions.

- Linear SVM → faster, but limited for complex fraud patterns
- RBF SVM → better performance on non-linear and imbalanced data

LAB 12: K-Means & Hierarchical Clustering

INTRODUCTION

Customer segmentation groups shoppers based on behavior using **unsupervised learning**.

We use:

- Hierarchical Clustering → to estimate number of clusters
- K-Means → to form final customer segments

Goal: Identify **high-value customer groups** for targeted marketing.

PROBLEM STATEMENT

Mall currently treats all customers equally → inefficient marketing.

Need to:

- Find hidden customer groups
- Use only **unlabeled data**
- Optimize marketing using spending behavior

DATASET DESCRIPTION

File: store_customers.csv

Features:

- CustomerID (drop)
- Gender (ignored)
- Age (ignored)
- Annual Income (k\$)
- Spending Score (1–100)

IMPORT LIBRARIES

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import silhouette_score
```

```
from scipy.cluster.hierarchy import dendrogram, linkage
```

LOAD + PREPROCESS DATA

```
df = pd.read_csv("store_customers.csv")
```

```
df = df.drop(columns=["CustomerID", "Gender", "Age"])
```

```
df.head()
```

FEATURE SELECTION

```
X = df[["Annual Income (k$)", "Spending Score (1-100)"]]
```

STANDARDIZATION

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

HIERARCHICAL CLUSTERING (DETERMINING K)

```
linked = linkage(X_scaled, method="ward")
```

```
plt.figure(figsize=(10,5))
```

```
dendrogram(linked)
```

```
plt.title("Dendrogram")
```

```
plt.show()
```

K-MEANS CLUSTERING

```
kmeans = KMeans(n_clusters=5, random_state=42)
```

```
df["Cluster"] = kmeans.fit_predict(X_scaled)
```

```
df.head()
```

VISUALIZATION OF CLUSTERS

```
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=df["Cluster"])
```

```
plt.xlabel("Annual Income")
```

```
plt.ylabel("Spending Score")
```

```
plt.title("Customer Segments")
```

```
plt.show()
```

SILHOUETTE SCORE (MODEL QUALITY)

```
score = silhouette_score(X_scaled, df["Cluster"])
```

```
print("Silhouette Score:", score)
```

CLUSTER REVENUE CONTRIBUTION

```
revenue = df.groupby("Cluster")["Annual Income (k$)"].sum()
```

```
total = df["Annual Income (k$)"].sum()
```

```
contribution = (revenue / total) * 100
```

```
print(contribution)
```

KPI FORMULAS

Silhouette Score

Measures cluster separation (-1 to 1)

Higher = better clustering

Revenue Contribution

$$(\text{cluster income} / \text{total income}) \times 100$$

CONCLUSION

K-Means successfully divides customers into **5 behavioral groups**.

- High income + high spending → premium customers
- Low income + low spending → low priority segment

Silhouette Score validates cluster quality, while revenue contribution identifies **most profitable customer groups**.

LAB 13: Outlier Detection

INTRODUCTION

Outlier detection identifies **abnormal data points** in banking datasets such as fraud, errors, or unusual customer behavior.

We use:

- Distance-based detection (optional concept)
- **Isolation Forest (main method)**

Goal: detect anomalies in financial records automatically.

PROBLEM STATEMENT

A bank currently uses manual rules → inefficient and inaccurate.

Need:

- Detect multivariate anomalies (e.g., low salary + high balance)
- Reduce false alarms
- Automate fraud/anomaly detection using ML

DATASET DESCRIPTION

File: Churn_Modelling.csv

Remove irrelevant columns:

- RowNumber
- CustomerId
- Surname

Important features:

- CreditScore
- Age
- Tenure
- Balance
- NumOfProducts
- EstimatedSalary

IMPORT LIBRARIES

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.ensemble import IsolationForest

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import confusion_matrix
```

LOAD + CLEAN DATA

```
df = pd.read_csv("Churn_Modelling.csv")

df = df.drop(columns=["RowNumber", "CustomerId", "Surname"])

df.head()
```

FEATURE SELECTION

```
X = df[[

    "CreditScore",

    "Age",

    "Tenure",

    "Balance",
```

```
"NumOfProducts",  
  
"EstimatedSalary"  
  
]]
```

NORMALIZATION

```
scaler = StandardScaler()  
  
X_scaled = scaler.fit_transform(X)
```

ISOLATION FOREST MODEL

```
model = IsolationForest(  
  
    contamination=0.05,  
  
    random_state=42  
  
)  
  
  
df["Anomaly"] = model.fit_predict(X_scaled)
```

Interpretation:

- -1 → Outlier (anomaly)

- 1 → Normal data

VIEW OUTLIERS

```
outliers = df[df["Anomaly"] == -1]
```

```
outliers.head()
```

COUNT ANOMALIES

```
print("Total Outliers:", len(outliers))
```

```
print("Total Normal:", len(df[df["Anomaly"] == 1]))
```

KPI CALCULATIONS

(A) If true labels exist (supervised evaluation case)

Assume:

```
y_true = df["True_Label"] # 1 = anomaly, 0 = normal
```

```
y_pred = np.where(df["Anomaly"] == -1, 1, 0)
```

Confusion Matrix

```
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
```

ANOMALY DETECTION RATE (RECALL)

```
anomaly_detection_rate = tp / (tp + fn)
```

```
print("Anomaly Detection Rate:", anomaly_detection_rate)
```

FALSE ALARM RATE

```
false_alarm_rate = fp / (fp + tn)
```

```
print("False Alarm Rate:", false_alarm_rate)
```

KPI FORMULAS

Anomaly Detection Rate

$TP / (TP + FN)$

False Alarm Rate

$FP / (FP + TN)$

CONCLUSION

Isolation Forest successfully detects abnormal banking behavior.

- Normal customers form dense patterns
- Outliers are isolated quickly using tree splits

Final system improves:

- Fraud detection capability
- Reduction of false alarms
- Automated financial risk monitoring

LAB 14: Data Scraping & Sentiment Analysis

INTRODUCTION

Sentiment analysis is used to classify customer opinions as:

- Positive
- Negative
- Neutral

We combine:

- Web scraping (data collection pattern)
- Text preprocessing
- VADER (lexicon-based sentiment)
- TF-IDF + Logistic Regression (supervised ML)

Goal: analyze customer feedback at scale.

DATASET

File: reviews.csv

- 600 synthetic product reviews

- Labels: positive / negative / neutral
- Contains noisy real-world style text

IMPORT LIBRARIES

```
import pandas as pd
```

```
import numpy as np
```

```
import re
```

```
import nltk
```

```
from nltk.corpus import stopwords
```

```
from nltk.sentiment import SentimentIntensityAnalyzer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score
```

WEB SCRAPING PATTERN (REFERENCE ONLY)

```
import requests
```

```
from bs4 import BeautifulSoup
```

```
url = "https://example.com/reviews"
```

```
response = requests.get(url)
```

```
soup = BeautifulSoup(response.text, "html.parser")
```

```
reviews = soup.find_all("p", class_="review")
```

```
raw_reviews = [r.text for r in reviews]
```

LOAD DATA

```
df = pd.read_csv("reviews.csv")
```

```
df.head()
```

TEXT CLEANING

```
nltk.download("stopwords")
```

```
stop_words = set(stopwords.words("english"))
```

```
def clean_text(text):
```

```
    text = text.lower()
```

```
    text = re.sub(r"http\S+|[\^a-z\s]", "", text)
```

```
    tokens = text.split()
```

```
    tokens = [w for w in tokens if w not in stop_words]
```

```
    return " ".join(tokens)
```

```
df["cleaned"] = df["review"].apply(clean_text)
```

VADER SENTIMENT ANALYSIS

```
nltk.download("vader_lexicon")
```

```
sia = SentimentIntensityAnalyzer()
```

```
def get_sentiment(text):
```

```

score = sia.polarity_scores(text)["compound"]

if score >= 0.05:

    return "positive"

elif score <= -0.05:

    return "negative"

else:

    return "neutral"

df["vader_sentiment"] = df["cleaned"].apply(get_sentiment)

```

VADER DISTRIBUTION

```
df["vader_sentiment"].value_counts()
```

SUPERVISED MODEL (TF-IDF + LOGISTIC REGRESSION)

Train-Test Split

```

X_train, X_test, y_train, y_test = train_test_split(

    df["cleaned"],

```

```
df["label"],  
  
test_size=0.2,  
  
random_state=42  
)
```

TF-IDF Feature Extraction

```
vectorizer = TfidfVectorizer(ngram_range=(1,2))  
  
X_train_tfidf = vectorizer.fit_transform(X_train)  
  
X_test_tfidf = vectorizer.transform(X_test)
```

Train Model

```
model = LogisticRegression(max_iter=1000)  
  
model.fit(X_train_tfidf, y_train)
```

Predictions

```
y_pred = model.predict(X_test_tfidf)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

KPI COMPARISON

VADER Accuracy (approx)

Agreement with true labels ≈ 0.76

Supervised Model Accuracy

TF-IDF + Logistic Regression $\approx 0.98+$

KEYWORD INSIGHTS (OPTIONAL VISUAL IDEA)

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
pos_words = " ".join(df[df["label"]=="positive"]["cleaned"])
```

```
neg_words = " ".join(df[df["label"]=="negative"]["cleaned"])
```

```
print(pos_words[:200])
```

```
print(neg_words[:200])
```

KPI SUMMARY

Metric	Method	Result
Sentiment Distribution	VADER labels	Positive / Neutral / Negative split
VADER Accuracy	Lexicon-based	~0.76
ML Accuracy	TF-IDF + LogReg	~0.98+
Best Model	Supervised ML	TF-IDF + Logistic Regression

CONCLUSION

- VADER is fast but limited (lexicon-based).
- TF-IDF + Logistic Regression learns dataset-specific patterns and performs significantly better.
- Web scraping pattern enables real-world review collection.
- Final system provides scalable sentiment analysis for business insights.

LAB 15: Performance Comparison of Models

INTRODUCTION

Hospitals need to predict whether a patient will be **readmitted within 30 days**.

Goal:

- Reduce readmission risk
- Improve healthcare quality
- Optimize resource allocation

We compare multiple ML models to find the best performer.

DATASET

UCI Diabetes 130-US Hospitals Dataset

- 100,000+ hospital records
- Features:
 - Demographics
 - Diagnoses
 - Medications

- Lab results
- Target:
 - Readmitted within 30 days (1 = Yes, 0 = No)

IMPORT LIBRARIES

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier,  
AdaBoostClassifier
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score
```

LOAD + PREPROCESS DATA

```
df = pd.read_csv("diabetes.csv")

df = df.drop(columns=["patient_nbr", "encounter_id"])

df = df.dropna()
```

TARGET CONVERSION

```
df["readmitted"] = df["readmitted"].apply(

    lambda x: 1 if x == "<30" else 0

)
```

FEATURES & TARGET

```
X = df.drop("readmitted", axis=1)

y = df["readmitted"]
```

ENCODING + SCALING

```
X = pd.get_dummies(X, drop_first=True)
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

TRAIN-TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X_scaled, y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

DEFINE MODELS

```
models = {
```

```
    "Logistic Regression": LogisticRegression(max_iter=1000),
```

```
"Decision Tree": DecisionTreeClassifier(),

"Random Forest": RandomForestClassifier(),

"KNN": KNeighborsClassifier(n_neighbors=7),

"Naive Bayes": GaussianNB(),

"SVM (RBF)": SVC(probability=True),

"Gradient Boosting": GradientBoostingClassifier(),

"AdaBoost": AdaBoostClassifier()

}
```

TRAIN + EVALUATE ALL MODELS

```
results = []
```

```
for name, model in models.items():
```

```
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
```

```
    # ROC-AUC requires probabilities
```

```

if hasattr(model, "predict_proba"):

    y_prob = model.predict_proba(X_test)[:,-1]

else:

    y_prob = model.decision_function(X_test)


results.append({

    "Model": name,

    "Accuracy": accuracy_score(y_test, y_pred),

    "Precision": precision_score(y_test, y_pred),

    "Recall": recall_score(y_test, y_pred),

    "F1": f1_score(y_test, y_pred),

    "ROC-AUC": roc_auc_score(y_test, y_prob)

})

```

```

results_df = pd.DataFrame(results)

```

```

results_df.sort_values(by="ROC-AUC", ascending=False)

```

ROC CURVE (ALL MODELS)

```
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import roc_curve
```

```
plt.figure()
```

```
for name, model in models.items():
```

```
    if hasattr(model, "predict_proba"):
```

```
        y_prob = model.predict_proba(X_test)[:,-1]
```

```
    else:
```

```
        y_prob = model.decision_function(X_test)
```

```
    fpr, tpr, _ = roc_curve(y_test, y_prob)
```

```
    plt.plot(fpr, tpr, label=name)
```

```
plt.plot([0,1],[0,1],"--")
```

```
plt.title("ROC Curves Comparison")
```

```
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
```

```
plt.legend()
```

```
plt.show()
```

BEST MODEL SELECTION

```
best_model = results_df.sort_values(by="ROC-AUC", ascending=False).iloc[0]
```

```
print(best_model)
```

KPI SUMMARY (EXPECTED INSIGHT)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	~0.75	~0.74	~0.75	~0.75	0.828 (Best AUC)
Gradient Boosting	0.763 (Best Acc.)	0.75	0.788	0.769	0.821
SVM	0.752	0.746	0.763	0.754	0.820
Others	lower	lower	lower	lower	lower

CONCLUSION

- Multiple ML models were compared for hospital readmission prediction.
- Logistic Regression achieved the **best ROC-AUC (0.828)**.
- Gradient Boosting achieved the **highest accuracy (0.763)**.